Universidade Leiria • Oeste
Escola de Tecnologia e Gestão

Algoritmos e Estruturas de Dados

2º ano, 1º semestre – 2026/2027
Ficha Prática 3

Carlos Urbano
Catarina Reis
João Ramos
José Magno
Marco Ferreira
Sónia Luz

Algoritmos de Ordenação

Os exercícios seguintes devem ser desenvolvidos nos packages:

- **aed.ordenacao.algoritmos**
- **aed.ordenacao.utilizacao**
- **aed.modelo.contactos**
- **aed.modelo.contactos.comparadores**

1. InsertionSort

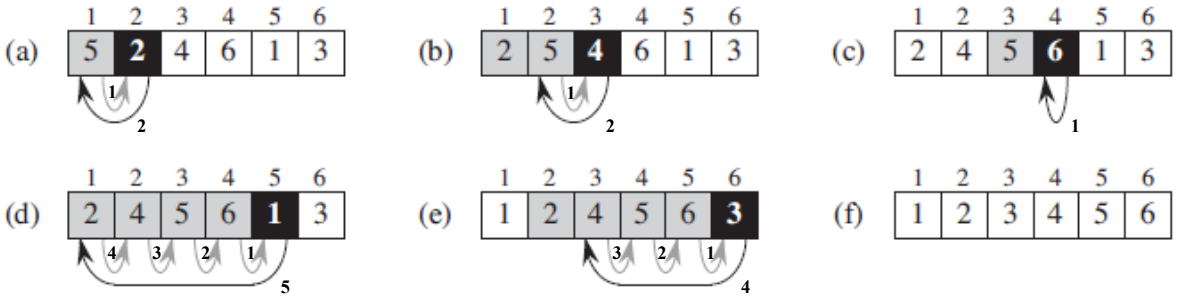


Figura 1. InsertionSort
(adaptado de http://www.java2novice.com/java-sorting-algorithms/insertion-sort/)

O algoritmo de ordenação InsertionSort (Figura 1) consiste em

1. Dividir uma sequência em duas partes:
   - Subsequência ordenada (parte esquerda, constituída inicialmente apenas pelo primeiro elemento);
   - A subsequência por ordenar (parte direita).

2. Em cada iteração, selecionar o primeiro elemento da subsequência por ordenar e percorrer os elementos da subsequência ordenada, a partir do último elemento e em direção ao primeiro. Enquanto esses elementos tiverem uma ordem superior ao elemento selecionado, coloca-os na posição seguinte (setas cinzentas na Figura 1). Quando o algoritmo encontra um elemento com ordem não superior ao elemento selecionado, encontra assim espaço para colocar o elemento selecionado na posição seguinte (setas pretas na Figura 1).

3. O algoritmo termina quando a subsequência por ordenar for vazia.

Carlos Urbano
Catarina Reis
João Ramos
José Magno
Marco Ferreira
Sónia Luz

Com base na especificação acima, resolva os exercícios seguintes:

a) Na classe **InsertionSort** implemente o método **ordenar** que execute um algoritmo iterativo, que produza o efeito da supracitado;

b) Simule no *jGRASP* a execução da classe **MainPraticaInsertionSort**.

2. Ordenação de Contactos

a) Implemente os critérios de **ComparacaoContactosPorNumeroTelefoneDes** e **ComparacaoContactosPorUltimoNomeAscSegPorDataNascimentoDes** no package **aed.modelo.contactos.comparadores**, que permitam ordenar os contactos, respetivamente, por:

- número de telefone, de forma descendente;
- último nome, de forma ascendente e, em caso de empate, seguidamente por data de nascimento de forma descendente.

b) Simule no *jGRASP* a execução da classe **MainPraticaOrdenacaoContactos** que contém os contactos apresentados na Tabela 1.

Primeiro Nome	Último Nome	Número de Telefone	Morada	Data de Nascimento
Ana	Silva	950000000	Rua de Leiria	01/10/1990
Ana	Rita	990000000	Travessa 25 de Abril	15/06/2000
Hugo	Santos	971234567	Avenida 1º de Maio	18/03/1994
Teresa	Silva	950000001	Rua de Leiria	02/10/1990
Eça	Queiroz	100000000	Praça do Almada	25/11/1845

Tabela 1. Contactos

Carlos Urbano
Catarina Reis
João Ramos
José Magno
Marco Ferreira
Sónia Luz

Algoritmos de Pesquisa

Os exercícios seguintes devem ser desenvolvidos nos packages:

- **aed.pesquisa.algoritmos**
- **aed.pesquisa.utilizacao**

3. Pesquisa Ternária

O algoritmo de pesquisa "Pesquisa Ternária" permite pesquisar um determinado elemento numa sequência ordenada e consiste em, repetidamente:

a) Dividir a sequência em três subsequências de tamanho semelhante (esquerda, central e direita);

b) Comparar o elemento a pesquisar com o elemento do primeiro ponto intermédio (limite superior da subsequência esquerda):

i. Caso seja de ordem inferior, considera na próxima iteração apenas a procura do elemento na subsequência esquerda;

ii. Caso seja de ordem superior, comparar o elemento a pesquisar com o elemento do segundo ponto intermédio (limite superior da subsequência central):

1. Caso seja de ordem inferior, considera na próxima iteração apenas a procura do elemento na subsequência central;
2. Caso seja de ordem superior, considera na próxima iteração apenas a procura do elemento na subsequência direita;
3. Caso seja igual, devolve a posição na sequência;

iii. Caso seja igual, devolve a posição na sequência.

c) O algoritmo termina quando a subsequência por pesquisar for vazia devolvendo, neste caso, uma indicação de que o elemento a procurar não pertence à sequência.

Carlos Urbano
Catarina Reis
João Ramos
José Magno
Marco Ferreira
Sónia Luz

Com base na especificação acima, resolva os exercícios seguintes:

a) Na classe **PesquisaTernariaRecursiva** implemente o método **pesquisar** que execute um algoritmo recursivo, que produza o efeito da supracitado;

b) Na classe **PesquisaTernariaIterativa** implemente o método **pesquisar** que execute um algoritmo iterativo, que produza o efeito da supracitado;

c) Simule no *jGRASP* a execução da classe **MainPraticaPesquisaTernaria**.